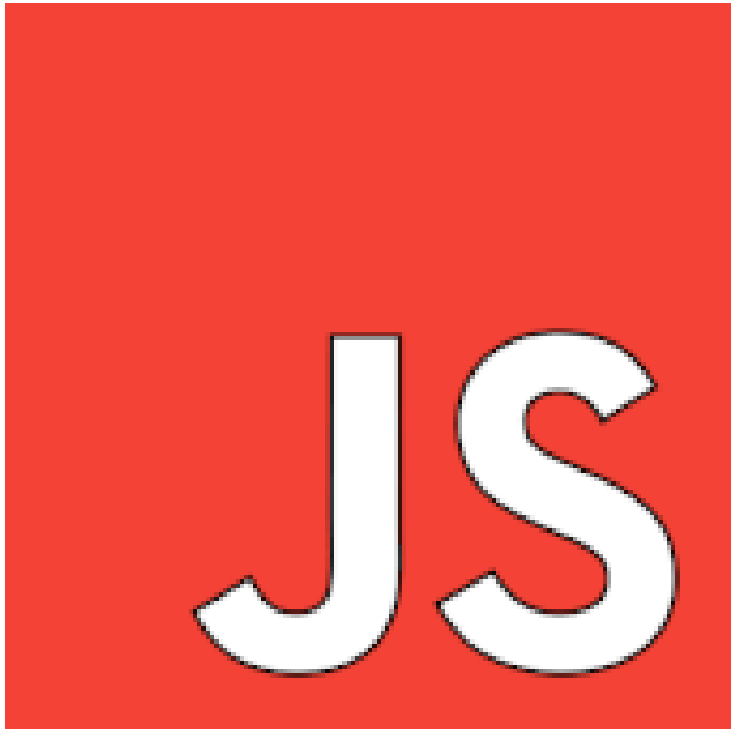


Unit-5

JavaScript

Dr. Falguni Suthar
Faculty of Computer Applications

Ms. Hiral Patel
Faculty of Computer Applications

Ganpat University
॥ विद्यया समाजोत्कर्षः ॥

Conditional Statements in JavaScript

What Are Conditional Statements?

Conditional statements allow you to control the flow of execution in your programs based on certain conditions.

They help in making decisions and running different blocks of code depending on whether a condition is true or false.

Types of Conditional Statements:

- Use `if` to specify a block of code to be executed, if a specified condition is true
- Use `else` to specify a block of code to be executed, if the same condition is false
- Use `else if` to specify a new condition to test, if the first condition is false
- Use `switch` to specify many alternative blocks of code to be executed

The if Statement

Use the `if` statement to specify a block of JavaScript code to be executed if a condition is true.

Syntax

```
if (condition) {
  //  block of code to be executed if the condition is true
}
```

Example:

```
<script>
var a=20;
if(a>10){
document.write("value of a is greater than 10");
}
</script>
```

The else Statement

Use the `else` statement to specify a block of code to be executed if the condition is false.

```
if (condition) {
  //  block of code to be executed if the condition is true
} else {
  //  block of code to be executed if the condition is false
}
```

Example:

```
<script>
var a=20;
if(a%2==0){
document.write("a is even number");
}
else{
document.write("a is odd number");
}
</script>
```

The else if Statement

Use the `else if` statement to specify a new condition if the first condition is false.

Syntax

```
if (condition1) {
  //  block of code to be executed if condition1 is true
} else if (condition2) {
  //  block of code to be executed if the condition1 is false and condition2 is true
} else {
  //  block of code to be executed if the condition1 is false and condition2 is false
}
```

Example:

```
<script>
var a=20;
if(a==10){
document.write("a is equal to 10");
}
else if(a==15){
document.write("a is equal to 15");
}
else if(a==20){
document.write("a is equal to 20");
}
else{
document.write("a is not equal to 10, 15 or 20");
}
</script>
```

The JavaScript Switch Statement

Use the `switch` statement to select one of many code blocks to be executed.

Syntax

```
switch(expression) {
  case x:
    // code block
    break;
  case y:
    // code block
    break;
  default:
    // code block
}
```

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript switch</h2>

<p id="demo"></p>

<script>
let day;
switch (new Date().getDay()) {
  case 0:
    day = "Sunday";
    break;
  case 1:
    day = "Monday";
    break;
  case 2:
    day = "Tuesday";
    break;
  case 3:
    day = "Wednesday";
    break;
  case 4:
    day = "Thursday";
    break;
  case 5:
    day = "Friday";
    break;
  case  6:
    day = "Saturday";
}
document.getElementById("demo").innerHTML = "Today is " + day;
</script>

</body>
</html>
```

The break Keyword

When JavaScript reaches a `break` keyword, it breaks out of the switch block.

This will stop the execution inside the switch block.

It is not necessary to break the last case in a switch block. The block breaks (ends) there anyway.

The default Keyword

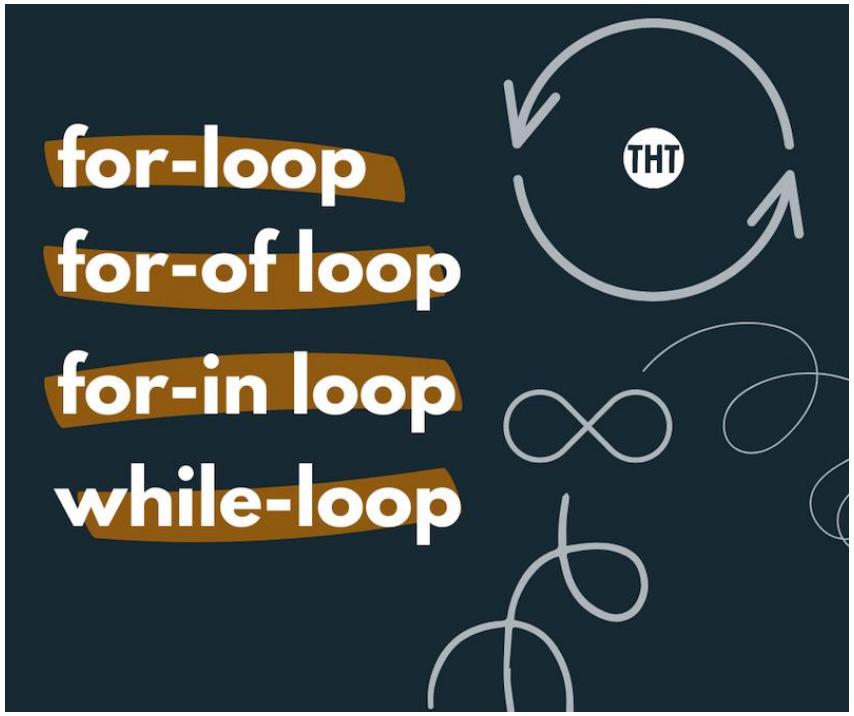
The `default` keyword specifies the code to run if there is no case match:

JavaScript Loops

The **JavaScript loops** are used *to iterate the piece of code* using for, while, do while or for-in loops. It makes the code compact. It is mostly used in array. It is also called iterative statements.

Advantages of using Loop:

1. It perform repetitive task with less number of codes.



Types of Loops

1) JavaScript For loop

The **JavaScript for loop** *iterates the elements for the fixed number of times*. It should be used if number of iteration is known. It executes a block of code until a specified condition is true.

The syntax of for loop is given below.

```
for (initialization; condition; increment)
{
   code to be executed
}
```

In the above syntax:

- **Initialize:** This is an expression that starts the loop.
- **condition:** This is a Boolean expression that specifies whether the loop should execute a loop or not.
- **Increment/Decrement:** Executes the iterate (i.e. increment and decrement) after each iteration of the for loop statement.

Example:

```
<script>
for (i=1; i<=5; i++)
{
document.write(i + "<br/>")
}
</script>
```

Output:

1

2

3

4

5

2) JavaScript while loop

The **JavaScript while loop** *iterates the elements for the infinite number of times*. It should be used if number of iteration is not known. In the while loop, the condition is first checked and if it evaluates to TRUE, the statements inside the curly braces are executed.

The syntax of while loop is given below.

```
while (condition)
{
    code to be executed
}
```

Example:

```
<script>
var i=11;
while (i<=15)
{
document.write(i + "<br/>");
i++;
}
</script>
```

Output:

11

12

13

14

15

3) JavaScript do while loop

The **JavaScript do while loop** *iterates the elements for the infinite number of times* like while loop. But, code is *executed at least* once whether condition is true or false. It is also called an exit controlled looping statement because the test condition is checked at the end of the do-while loop and hence the body is executed unconditionally for the first time.

The syntax of do while loop is given below.

```
do{
    code to be executed
}while (condition);
```

Example:

```
<script>
var i=21;
do{
document.write(i + "<br/>");
i++;
}while (i<=25);
</script>
```

Output:

21

22

23

24

25

4) JavaScript for in loop

The **JavaScript for in loop** is used *to iterate the properties of an object*. JavaScript provides a variation of a loop known as a for loop that can be used to extract the names and values of any object. It allows looping through all elements of an array or properties of an object.

The syntax of for in loop is given below.

```
for ( v_name in object_name)
{
statements(s);
}
```

In the above syntax, v_name is the name of the variable and object_name is the name of the object.

Example:

```
<!DOCTYPE html>
<html>
<body>
<h2> Example of For In Loops</h2>
<p id="data"> </p>
<script>
const numbers = [75, 14, 89, 16];
let text = " ";
for (i in numbers) {
  text += numbers[i] + "<br>";
}
document.getElementById("data").innerHTML = text;
</script>
</body>
</html>
```

Output:

Example of For In Loops

75

14

89

16

JavaScript Functions

JavaScript functions are used to perform operations. We can call JavaScript function many times to reuse the code.

Advantage of JavaScript function

Functions are useful in organizing the different parts of a script into the several tasks that must be completed. There are mainly two advantages of JavaScript functions.

1. **Code reusability:** We can call a function several times in a script to perform their tasks so it saves coding.
2. **Less coding:** It makes our program compact. We don't need to write many lines of code each time to perform a common task.

Rules for naming functions:

- It must be case sensitive.
- It must be start with alphabetical character (A-Z) or an underscore symbol.
- It cannot contain spaces.
- It cannot be use as a reserve words.

How to declare a Function:

To declare a function we have to use the reserved keyword "function", followed by its name, and a set of arguments.

JavaScript Function Syntax

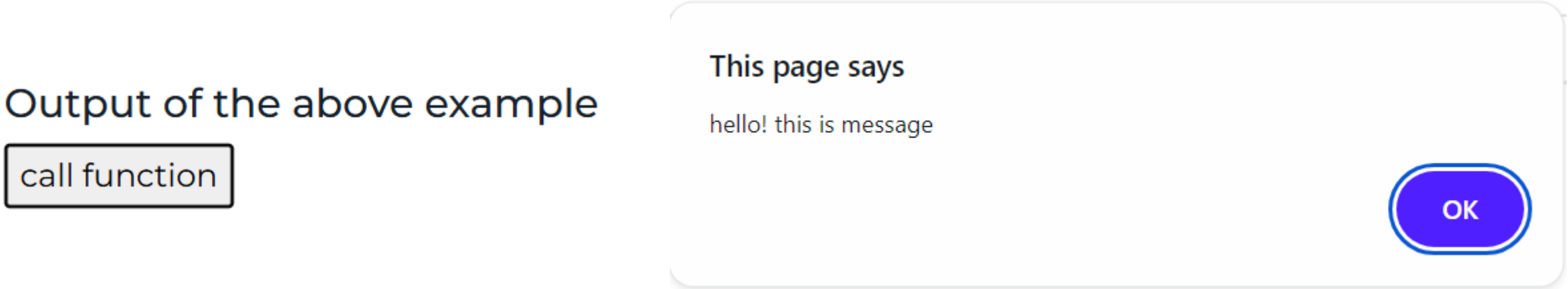
The syntax of declaring function is given below.

```
function functionName([arg1, arg2, ...argN]){
 //code to be executed
}
```

Example:

```
<script>
function msg(){
alert("hello! this is message");
}
</script>
<input type=button onclick=msg() value="call function"/>
```

Output of the above example



JavaScript Function Arguments

```
<script>
function getcube(number){
alert(number*number*number);
}
</script>
<form>
<input type=button value=click onclick=getcube(4) />
</form>
```

Output of the above example



Function Expression

The Function expression in JavaScript allows you to define a function as an expression. The function expression is similar to the anonymous function declaration. The function expression can be assigned to a varaible.

The syntax of function expression in JavaScript is as follows–

```
const varName = function (parameter-list) {
    statements
};
```

In the example below, we have defined a JavaScript function using function expression and assigned it to a vriable name myFunc.

```
const myFunc = function (x, y){
   return x + y;
};
```

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Functions</h2>
<p>After a function has been stored in a variable,
the variable can be used as a function:</p>

<p id="demo"></p>

<script>
const x = function (a, b) {return a * b};
document.getElementById("demo").innerHTML = x(4, 3);
</script>

</body>
</html>
```

JavaScript Functions

After a function has been stored in a variable, the variable can be used as a function:

12

JavaScript Form Validation

JavaScript Form Validation is a way to ensure that the data users enter into a form is correct before it gets submitted. This helps ensure that things like emails, passwords, and other important details are entered properly, making the user experience smoother and the data more accurate.

Example:

file:///D:/2024-2025/Sem-2/AWP/Practical%20Examples/functioneg.html

Steps for Form Validation in JavaScript

When we validate a form in JavaScript, we typically follow these steps:

- **Data Retrieval:**
    - The first step is to get the user's values entered into the form fields (like name, email, password, etc.). This is done using document.forms.RegForm, which refers to the form with the name "RegForm".

- **Data Validation:**
    - Name Validation: We check to make sure the name field isn't empty and doesn't contain any numbers.
    - Address Validation: We check that the address field isn't empty.
    - Email Validation: We make sure that the email field isn't empty and that it includes the "@" symbol.
    - Password Validation: We ensure that the password field isn't empty and that the password is at least 6 characters long.
    - Course Selection Validation: We check that a course has been selected from a dropdown list.
- **Error Handling:**
    - If any of the checks fail, an alert message is shown to the user using window.alert, telling them what's wrong.
    - The form focuses on the field that needs attention, helping the user easily fix the error.

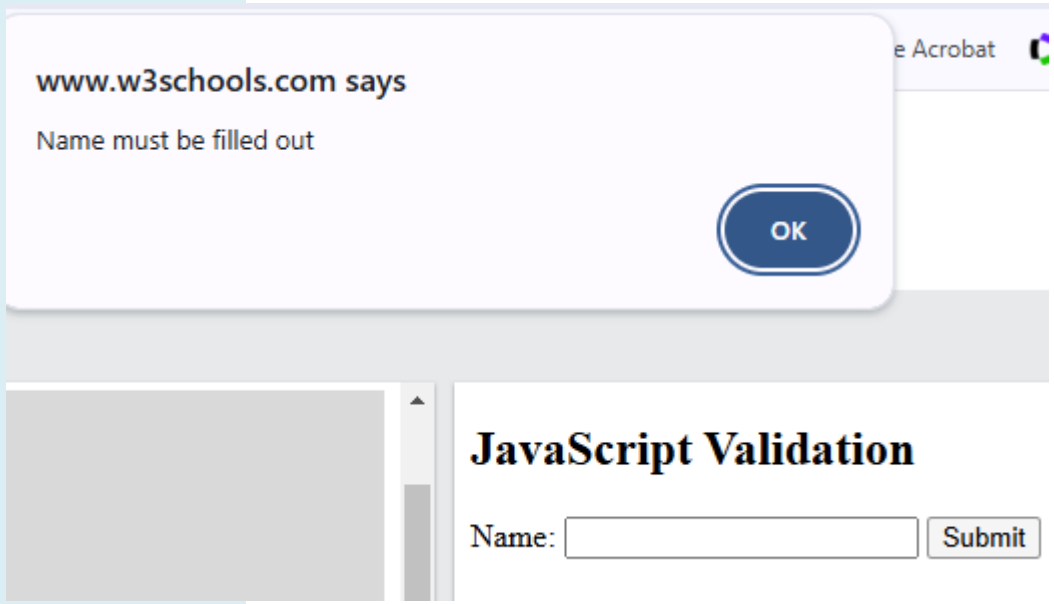
- **Submission Control:**
    - If all the validation checks pass, the function returns true, meaning the form can be submitted. If not, it returns false, stopping the form from being submitted.
- **Focus Adjustment:**
    - The form automatically focuses on the first field that has an error, guiding the user to fix it.

```
<!DOCTYPE html>
<html>
<head>
<script>
function validateForm() {
  let x = document.forms["myForm"]["fname"].value;
  if (x == "") {
    alert("Name must be filled out");
    return false;
  }
}
</script>
</head>
<body>

<h2>JavaScript Validation</h2>

<form name="myForm" action="/action_page.php" onsubmit="return
validateForm()" method="post">
  Name: <input type="text" name="fname">
  <input type="submit" value="Submit">
</form>
</body>
</html>
```

Output



```
<p>Please input a number between 1 and 10:</p>
<input id="numb">
<button type="button" onclick="myFunction()">Submit</button>
<p id="demo"></p>
<script>
function myFunction() {
  // Get the value of the input field with id="numb"
  let x = document.getElementById("numb").value;
  // If x is Not a Number or less than one or greater than 10
  let text;
  if (isNaN(x) || x < 1 || x > 10) {
    text = "Input not valid";
  } else {
    text = "Input OK";
  }
  document.getElementById("demo").innerHTML = text;
}
</script>
</body>
</html>
```

Types of Form Validation

- **Client-side Validation:**
    - This is done in the user's browser before the form is submitted. It provides quick feedback to the user, helping them fix errors without sending data to the server first.

- **Server-side Validation:**
    - Even though client-side validation is useful, it's important to check the data again on the server. This ensures that the data is correct, even if someone tries to bypass the validation in the browser.

Example for password validation:

file:///D:/2024-2025/Sem-2/AWP/Practical%20Examples/passval.html

How to validate confirm password using JavaScript ?

file:///D:/2024-2025/Sem-2/AWP/Practical%20Examples/confpassval.html

References

https://www.javatpoint.com/javascript-form-validation

https://www.tutorialspoint.com/javascript/index.htm

https://www.w3schools.com/js/default.asp

https://www.geeksforgeeks.org/javascript/?ref=dhm


Thank
you

Ganpat University

॥ विद्यया समाजोत्कर्षः ॥

